

CSCE 629 ANALYSIS OF ALGORITHMS · MODULE 12 · LECTURE NOTES

NP-Completeness and Reductions

Knowing when to stop looking for an exact efficient algorithm.

MODULE OUTCOMES

- Define P, NP, NP-hard, and NP-complete precisely.
- Explain what a polynomial-time reduction is and in which direction it runs.
- Prove a problem NP-complete by reduction from a known one.
- Name the standard starting problems and their typical uses.
- Explain what NP-completeness does and does not rule out.

1 The classes

A **decision problem** has a yes/no answer. The theory is stated for decision problems because it makes the definitions clean; optimisation versions are handled by asking 'is there a solution of value at least k?'

Class	Definition	Examples
P	Decidable in polynomial time.	Shortest path, bipartite matching, MST, linear programming.
NP	A proposed solution (a <i>certificate</i>) can be verified in polynomial time.	SAT, clique, Hamiltonian cycle, subset sum, graph colouring.
NP-hard	At least as hard as every problem in NP, via polynomial reduction. Need not itself be in NP.	TSP optimisation, the halting problem (which is not in NP at all).
NP-complete	In NP and NP-hard. The hardest problems that are still verifiable.	SAT, 3-SAT, vertex cover, clique, subset sum.

THE DEFINITION PEOPLE GET WRONG

NP does **not** stand for 'not polynomial'. It stands for *nondeterministic polynomial*, and the useful definition is about verification: given a candidate answer, can you check it quickly?

Given a proposed satisfying assignment for a Boolean formula, you substitute and evaluate — linear time. Given a proposed Hamiltonian cycle, you check it visits each vertex once. That is what membership in NP means.

$P \subseteq NP$ trivially: if you can find a solution quickly you can certainly check one. Whether the inclusion is strict is the P versus NP question, open since 1971 and believed by nearly everyone to be strict.

2 Reductions

A **polynomial-time reduction** from A to B, written $A \leq_p B$, is a polynomial-time transformation taking any instance of A to an instance of B with the same yes/no answer.

The meaning is: **A is no harder than B**. If you had a fast algorithm for B, you would have one for A — transform, then call it.

THE DIRECTION, AND WHY EVERYONE GETS IT BACKWARDS

To prove **your** problem B is hard, you must reduce a *known hard* problem A **into** B. The logic: A is hard; if B were easy, A would be easy; therefore B is not easy.

Reducing B to a known hard problem proves nothing whatsoever — every problem in NP reduces to SAT, including trivial ones.

Mnemonic: known hard problem → your problem. The arrow points at the thing you are proving hard. Say it out loud before writing any reduction; this error accounts for most wrong proofs in this material.

2.1 The recipe

- 1 Show $B \in \text{NP}$ by describing a certificate and a polynomial-time verifier. (Needed for *completeness*; skip it if you only want NP-hardness.)
- 2 Choose a known NP-complete problem A, preferably one whose structure resembles B.
- 3 Give a polynomial-time transformation from an arbitrary instance of A to an instance of B.
- 4 Prove the equivalence **in both directions**: the A-instance is a yes if and only if the constructed B-instance is a yes. Omitting a direction is the second most common error after getting the direction of the reduction wrong.

2.2 Where the chain starts

Reductions establish relative hardness, so something must be proved hard from first principles. **Cook and Levin** did this independently around 1971 for SAT: given any polynomial-time verifier, they showed how to construct a Boolean formula that is satisfiable exactly when the verifier accepts some certificate. Every problem in NP therefore reduces to SAT.

Karp followed with reductions from SAT to 21 well-known combinatorial problems, which turned an abstract theorem into a practical catalogue. Today thousands of problems are known to be NP-complete, and essentially all of them trace back through chains of reductions to Cook–Levin.

3 A worked reduction

Independent set $\leq_p$ vertex cover. An independent set is a set of vertices no two of which are adjacent. A vertex cover is a set touching every edge.

Claim. S is an independent set of G if and only if $V - S$ is a vertex cover of G .

($\Rightarrow$) Let S be independent and let (u,v) be any edge. Both u and v cannot lie in S , since S contains no edge. So at least one lies in $V - S$, meaning $V - S$ covers that edge. As the edge was arbitrary, $V - S$ is a vertex cover.

($\Leftarrow$) Let C be a vertex cover and take $u, v \in V - C$. If (u,v) were an edge, C would need to contain u or v , contradicting their membership in $V - C$. So no edge lies within $V - C$, which is therefore independent. ■

The reduction follows immediately: 'does G have an independent set of size at least k ?' becomes 'does G have a vertex cover of size at most $n - k$?'. The transformation is the identity on the graph and arithmetic on the parameter — linear time, and both directions are proved.

3.1 Choosing what to reduce from

Start from	When your problem involves
3-SAT	Anything. The default choice, and the structure (variables and clauses) is flexible enough to encode most things.
Vertex cover	Selecting a small set of elements that must touch or cover everything.
Clique / Independent set	Mutual compatibility or mutual conflict among items.
Hamiltonian cycle / path	Sequencing, routing, visiting things exactly once. The natural source for TSP-like problems.
Subset sum / Partition	Numbers, weights, capacities, packing, scheduling by duration.
3-colouring	Assigning limited resources under conflict constraints: register allocation, frequency assignment, exam timetabling.

The practical advice is to pick the source problem whose *structure* most resembles your target. Reductions between structurally similar problems are short; reductions across a structural gap require gadgets and get painful.

4 What NP-completeness licenses

WHAT IT DOES MEAN

No polynomial-time exact algorithm is known, and finding one would prove $P = NP$ and resolve the most famous open problem in the field.

So you may stop looking, and you may say so in a design review without embarrassment. That is a genuinely useful professional outcome: it converts an open-ended search into a bounded engineering decision.

WHAT IT DOES NOT MEAN

It does not mean your instances are hard. NP-completeness is a worst-case statement over all possible inputs. Modern SAT solvers routinely dispatch industrial instances with millions of variables, because those instances have structure that adversarial ones do not.

It does not mean no good algorithm exists. It rules out an exact polynomial algorithm correct on *every* input. Approximation algorithms with proven ratios, heuristics that work well in practice, and exact algorithms that are fast on realistic sizes are all still available.

Treating 'NP-complete' as 'impossible' is a real and costly error. It leads people to abandon approaches that would have worked.

4.1 The five responses

Strategy	What you give up	What you keep	Example
Approximation	Optimality	A <i>proven</i> bound on how far off you can be	2-approximation for vertex cover (Module 13)
Heuristics	Any guarantee at all	Often excellent results on real data	Simulated annealing, local search, genetic algorithms
Exact exponential	Scalability	Guaranteed optimality on small instances	Held–Karp TSP at $O(2^n n^2)$, branch and bound
Parameterised	Efficiency in general	Polynomial when some parameter k is small	Vertex cover in $O(2^k \cdot n)$
Restrict the input	Generality	Genuine polynomial time on your actual inputs	Many problems are easy on trees, planar, or bounded-treewidth graphs

THE LAST ROW DESERVES MORE ATTENTION THAN IT GETS

A great many NP-hard problems are polynomial on restricted graph classes: independent set on trees, colouring on interval graphs, almost everything on bounded treewidth.

Real instances frequently have that structure without anyone having checked. Dependency graphs are usually close to trees; geographic networks are planar; scheduling conflicts often have bounded overlap.

Before settling for an approximation, find out what is actually true of your inputs. Sometimes the answer is that your special case was never hard in the first place.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
MIT 6.046J — Complexity, Reductions, NP-completeness	Three lectures covering the definitions, the reduction technique, and several worked hardness proofs.
Jeff Erickson, Algorithms — NP-Hardness chapter	The clearest free treatment, with an unusually good discussion of how to pick which problem to reduce from and many worked gadgets.
Karp — Reducibility Among Combinatorial Problems (1972, free)	The original 21 reductions. Short, readable, and historically the paper that made the theory practical.
Complexity Zoo	A catalogue of complexity classes, for when you encounter one you do not recognise.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Prove vertex cover is NP-complete by reduction from independent set, writing out membership in NP and both directions of the equivalence.
- 2 Prove clique is NP-complete by reduction from independent set via the complement graph.
- 3 Reduce 3-SAT to independent set. This one needs a gadget: a triangle per clause plus edges between contradictory literals. Prove both directions.
- 4 Write a reduction in the *wrong* direction for some problem and explain in one paragraph exactly why it proves nothing.
- 5 Implement a brute-force exact solver and a 2-approximation for vertex cover. Compare on random graphs, and find the instance size at which exact becomes infeasible.

- 6 Take a problem from your own work that you suspect is hard. Either reduce a known NP-complete problem to it, or find a polynomial algorithm. Report which, with the argument.
- 7 Find an NP-hard problem that becomes polynomial on trees, implement both, and measure the difference.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define P and NP. What does the N stand for, and what is the useful working definition of NP?
- 2 What does $A \leq_p B$ mean? To prove B hard, which direction do you reduce, and why?
- 3 List the four steps of an NP-completeness proof, and name the two most common mistakes.
- 4 What did Cook and Levin prove, and why was a direct proof necessary?
- 5 Give two things NP-completeness does *not* tell you about your specific problem instances.
- 6 Name the five standard responses to an NP-hard problem, with what each sacrifices.